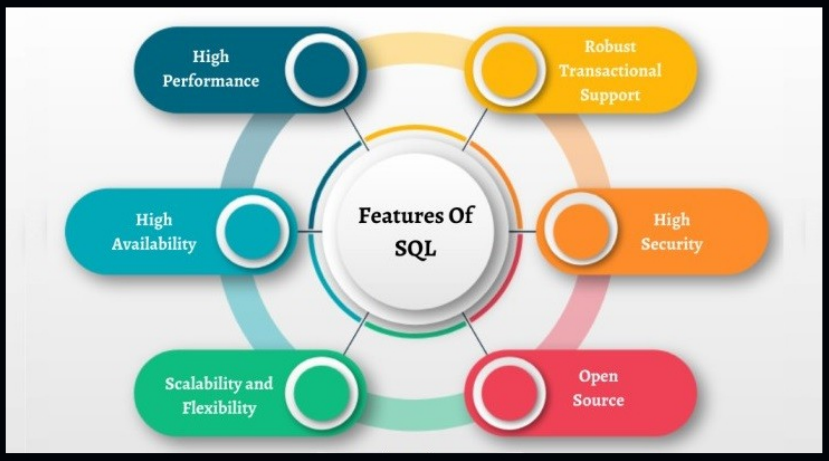
alone3000 / Coding
Type / to search
Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings
main Coding / Notes / sqlnotes / sql.md
Go to file
MrAlone1 k
ab40165 · 2 months ago History
Preview Code | Blame 955 lines (749 loc) · 27.3 KB Raw

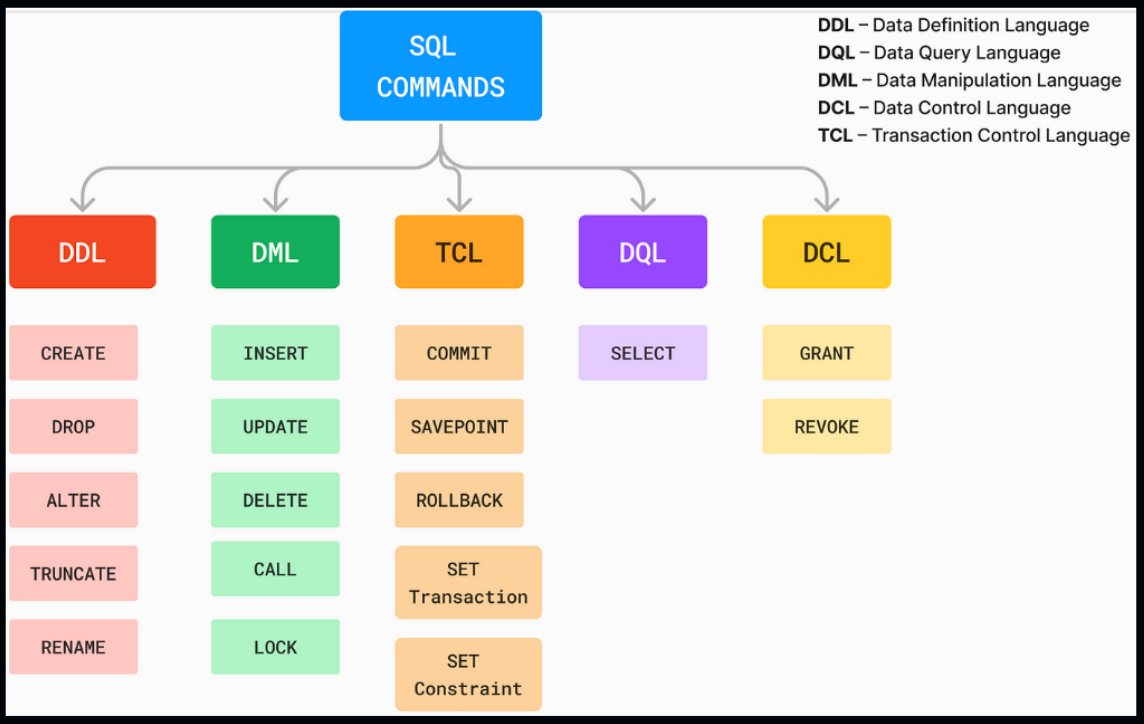
Introduction to SQL



SQL (Structured Query Language) is a standard language for managing and manipulating databases. It allows you to interact with a database system to perform various operations.

Types of SQL statements

- **DML (Data Manipulation Language)**: Used to perform operations like CREATE, INSERT, UPDATE, and DELETE.
- **DQL (Data Query Language)**: Used to retrieve data from the database.
- **DDL (Data Definition Language)**: Used to create, modify, and delete database structures
- **DCL (Data Control Language)**: Used to control access to the database.



setup environment steps

1. Install a database management system (DBMS) such as MySQL, PostgreSQL, or SQL Server
2. Create a new database
3. Connect to the database using a SQL client or IDE

4. Write and execute SQL queries to perform various operations.

user account CRUD operation in mysql server using command with example

```
-- Create a new user account
CREATE USER 'newuser'@'%' IDENTIFIED BY 'password';
-- Grant privileges to the new user account
GRANT ALL PRIVILEGES ON *.* TO 'newuser'@'%';
-- Update the user account
UPDATE mysql.user SET Password = PASSWORD('newpassword') WHERE User = 'newuser';
```

1. GRANT

Definition: The GRANT command is used to give users or roles permission to perform specific actions on database objects.

Syntax:

```
GRANT privilege_type ON object TO user_or_role;
```

Parameters:

`privilege_type`: The type of access you are granting (e.g., SELECT, INSERT, UPDATE, DELETE).

`object`: The database object (e.g., table, view).

`user_or_role`: The user or role to whom the privileges are being granted.

Examples:

Granting SELECT and INSERT privileges on the 'employees' table to a user 'john_doe':

```
GRANT SELECT, INSERT ON employees TO john_doe;
```

Granting all privileges on the 'orders' table to a role 'sales_team':

```
GRANT ALL PRIVILEGES ON orders TO sales_team;

GRANT CREATE ON *.* TO 'alex'@'localhost';
```

2. REVOKE

Definition: The REVOKE command is used to remove previously granted permissions from users or roles.

Syntax:

```
REVOKE privilege_type ON object FROM user_or_role;
```

Parameters:

Examples:

Revoking SELECT privilege on the 'employees' table from the user 'john_doe':

```
REVOKE SELECT ON employees FROM john_doe;
```

Revoking all privileges on the 'orders' table from the role 'sales_team':

```
REVOKE ALL PRIVILEGES ON orders FROM sales_team;
```

Verifying the Privileges

To check the privileges of a user, you can use:

```
SHOW GRANTS FOR 'alex'@'%';
```

Use of %

When you create a MySQL user, you can specify where the user can connect from:

```
1. If you use 'alex'@'localhost', the user can connect only from the local machine (localhost).
2. If you use 'alex'@'192.168.1.100', the user can connect only from a specific IP address (e.g., 192.168.1.100).
```

3. If you use 'alex'@'%', the user can connect from any host or IP address.

Basic Commands of SQL

SQL commands are used to perform various operations on a database. Here are some basic commands:

```
-- Create a new database
CREATE DATABASE database_name;
-- Use a Database
USE database_name;
-- Describe Schema
DESC table_name;
```

CRUD Operations

CREATE

Used to create new tables or databases.

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    ...
);
```

INSERT

Used to insert data into tables.

```
INSERT INTO table_name(column1,column2,...) value(value1,value2,...);

INSERT INTO table_name value(value1,value2,...); -- Maintaine same order of table fields

-- Multiple records insert into table
INSERT INTO table_name(column1,column2,...) value(value1,value2,...),
                                                 (value1,value2,...),
                                                 (value1,value2,...);
```

READ

Used to retrieve data from tables.

```
SELECT column1, column2 FROM table_name;

SELECT * FROM table_name; -- Retrieves all columns

SELECT column1, column2 FROM table_name WHERE condition;
```

UPDATE

- Don't forget to change safe update setting in edit menu inside SQL Editor

Used to modify existing records.

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

DELETE

Used to remove records.

```
DELETE FROM table_name
WHERE condition;

-- delete all record inside table
TRUNCATE TABLE table_name;
```

ALTER

```
ALTER TABLE table_name
ADD COLUMN new_column_name data_type;

ALTER TABLE table_name
DROP COLUMN column_name;

ALTER TABLE table_name
MODIFY COLUMN column_name data_type;
```

```sql
ALTER TABLE table_name
RENAME COLUMN old_column_name TO new_column_name;
--or
ALTER TABLE table_name CHANGE COLUMN old_column_name new_column_name data_type;

ALTER TABLE table_name
RENAME TO new_table_name;

-- Example
ALTER TABLE employees
ADD COLUMN email VARCHAR(255);

ALTER TABLE employees
DROP COLUMN phone_number;

ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(10, 2);

ALTER TABLE employees
RENAME COLUMN emp_id TO employee_id;

ALTER TABLE employees
RENAME TO staff;

ALTER TABLE table_name ADD PRIMARY KEY (column_name);

ALTER TABLE table_name ADD CONSTRAINT fk_column_name FOREIGN KEY (column_name) REFERENCES other_table_name(other_column_name);
```

Task

```
Assume a table student exist with cols ` rollno , name , age , contact , address ` with apropriate dataTypes

1. insert 10 new records
2. delete record with rollno = 101
3. update name of student with rollno = 102 to 'John'
4. delete all records from table
5. add a new column 'email' of type varchar(255) to the table
```

Constraints in SQL

Constraints are used to limit the type of data that can be inserted into a table.

- They help to maintain the integrity of the data in the database.

Types of Constraints:

```
1. PRIMARY KEY Constraint
```

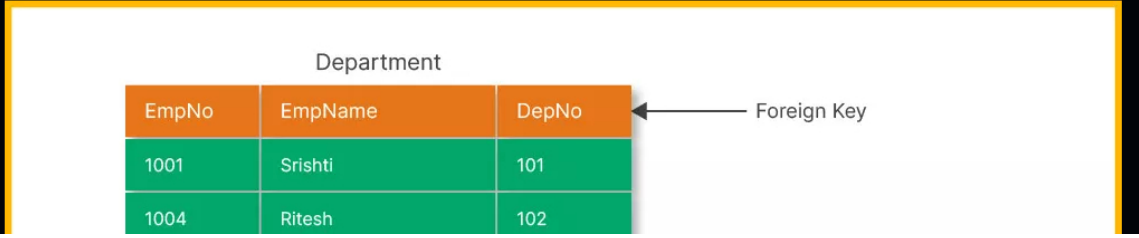
The PRIMARY KEY constraint uniquely identifies each record in a table. It ensures that no duplicate values are entered in the primary key column.

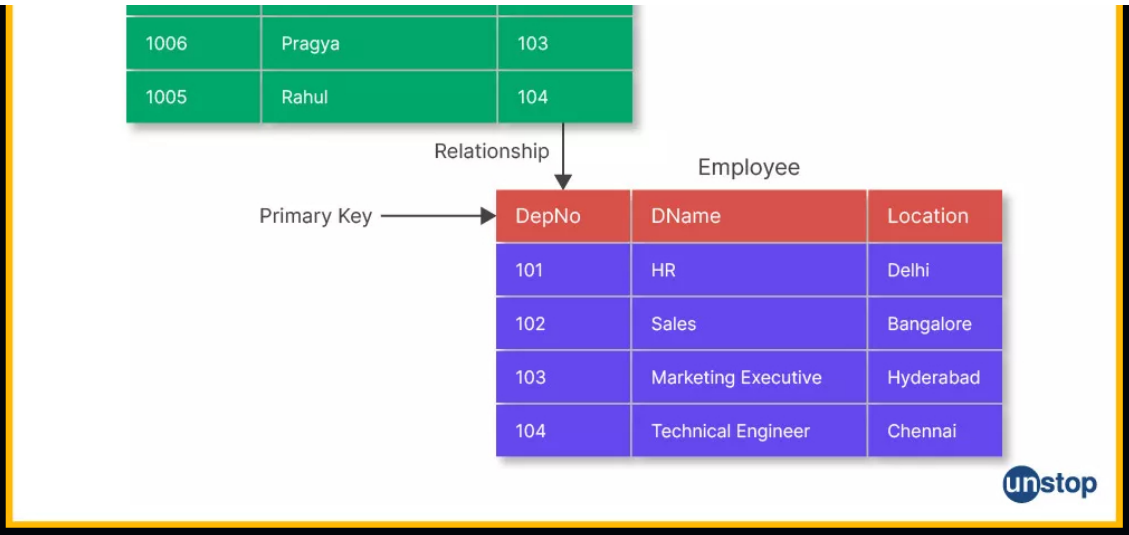
```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    email VARCHAR(255)
);
```

```
2. FOREIGN KEY Constraint
```

The FOREIGN KEY constraint is used to link two tables together. It ensures that the relationship between the two tables is valid.

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```





3. UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are unique. It prevents duplicate values from being entered in the column.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    email VARCHAR(255) UNIQUE
);
```

4. NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot contain null values. It prevents null values from being entered in the column.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255) NOT NULL,
    email VARCHAR(255)
);
```

5. CHECK Constraint

The CHECK constraint ensures that all values in a column meet a specific condition. It prevents invalid values from being entered in the column.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    age INT CHECK (age > 18)
);
```

6. DEFAULT Constraint

The DEFAULT constraint sets a default value for a column when no value is specified. It ensures that the column always has a value.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    country VARCHAR(255) DEFAULT 'USA'
);
```

7. INDEX Constraint

The INDEX constraint is used to improve the speed of data retrieval. It creates an index on a column or set of columns.

```
CREATE INDEX idx_customer_name ON customers (customer_name);
```

8. AUTO_INCREMENT Constraint

The AUTO_INCREMENT constraint is used to automatically increment the value of a column. It is commonly used for primary key columns.

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(255),
    email VARCHAR(255)
);
```

Alter Constraints

```sql
-- add constraint of 'not null' to column
ALTER TABLE employees
MODIFY COLUMN employee_name VARCHAR(255) NOT NULL;
-- remove constraint of 'not null' from column
ALTER TABLE employees
MODIFY COLUMN employee_name VARCHAR(255);

-- add constraint using ADD keyword
ALTER TABLE employees
ADD CONSTRAINT chk_age CHECK (age > 18);

-- remove constraint using DROP keyword
ALTER TABLE employees
DROP CONSTRAINT chk_age;

-- remove multiple constraints
ALTER TABLE employees
DROP CONSTRAINT chk_age,
DROP CONSTRAINT chk_salary;
```

Task

```
1. Create a table named `employees` with the following columns:
- `employee_id` (primary key, auto-incrementing integer)
- `employee_name` (varchar(255))
- `department` (varchar(255))
- `salary` (integer, default value 50000)

2. Change `employee_name` constraint to not null.

3. remove all constraints
```

SQL Operators

SQL operators are used to perform operations on data in a database. Here are some common SQL operators:

Comparison Operators

Comparison operators are used to compare values.

Operator	Description	Example
`=`	Equal to	`SELECT * FROM employees WHERE salary = 50000;`
`<>`	Not equal to	`SELECT * FROM employees WHERE salary <> 50000;`
`>`	Greater than	`SELECT * FROM employees WHERE salary > 50000;`
`<`	Less than	`SELECT * FROM employees WHERE salary < 50000;`
`>=`	Greater than or equal to	`SELECT * FROM employees WHERE salary >= 50000;`
`<=`	Less than or equal to	`SELECT * FROM employees WHERE salary <= 50000;`

Logical Operators

Logical operators are used to combine conditions.

Operator	Description	Example
`AND`	True if both conditions are true	`SELECT * FROM employees WHERE salary > 50000 AND department = 'Sales';`
`OR`	True if either condition is true	`SELECT * FROM employees WHERE salary > 50000 OR department = 'Marketing';`
`NOT`	True if the condition is false	`SELECT * FROM employees WHERE NOT department = 'Sales';`

Pattern Matching Operators

Pattern matching operators are used to search for patterns in strings.

Operator	Description	Example

`LIKE`	True if the string matches the pattern	`SELECT * FROM employees WHERE name LIKE '%John%';`
`NOT LIKE`	True if the string does not match the pattern	`SELECT * FROM employees WHERE name NOT LIKE '%John%';`
`REGEXP`	True if the string matches the regular expression	`SELECT * FROM employees WHERE name REGEXP 'John.*';`

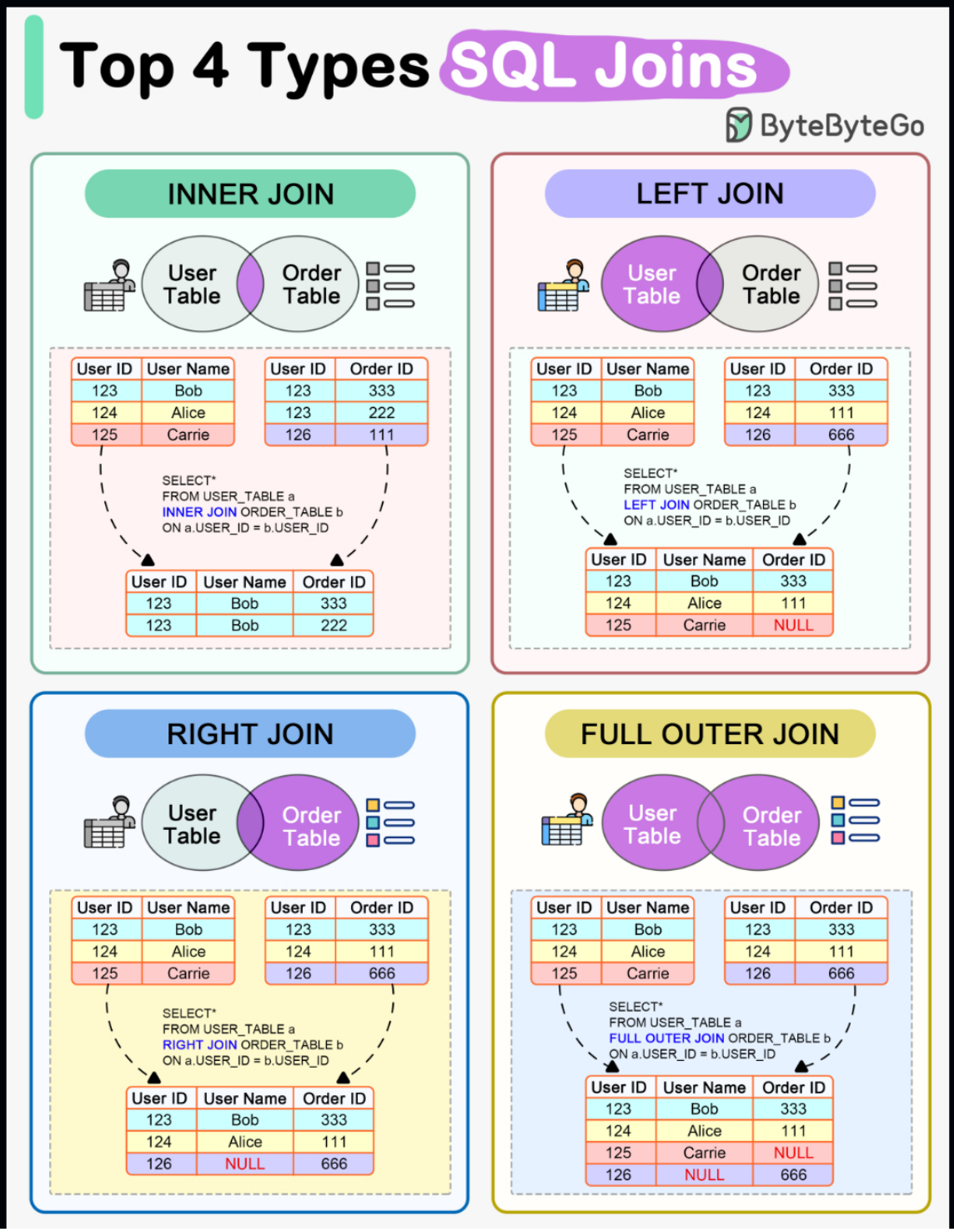
NULL Operators

NULL operators are used to test for NULL values.

Operator	Description	Example
`IS NULL`	True if the value is NULL	`SELECT * FROM employees WHERE department IS NULL;`
`IS NOT NULL`	True if the value is not NULL	`SELECT * FROM employees WHERE department IS NOT NULL;`

Joins

Joins are used to combine rows from two or more tables.



INNER JOIN

Retrieves records with matching values in both tables.

```
SELECT columns
FROM table1
INNER JOIN table2 ON table1.column = table2.column;
```

LEFT JOIN

Retrieves all records from the left table and matched records from the right table.

```
SELECT columns
FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

RIGHT JOIN

Retrieves all records from the right table and matched records from the left table.

```
SELECT columns
FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

FULL JOIN

Retrieves all records when there is a match in one of the tables.

```
SELECT columns
FROM table1
FULL JOIN table2 ON table1.column = table2.column;

-- mysql syntax

SELECT *
FROM table1
LEFT JOIN table2 ON table1.column = table2.column

UNION

SELECT *
FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Joining Multiple Tables

```
SELECT *
FROM table1
JOIN table2 ON table1.column = table2.column
JOIN table3 ON table2.column = table3.column;
```

```
1. From the following tables write a SQL query to find the salesperson and customer who reside in the same city. Return Salesma

Sample table: salesman

 salesman_id |    name    |   city   | commission
-------------+------------+----------+------------
        5001 | James Hoog | New York |       0.15
        5002 | Nail Knite | Paris    |       0.13
        5005 | Pit Alex   | London   |       0.11
        5006 | Mc Lyon    | Paris    |       0.14
        5007 | Paul Adam  | Rome     |       0.13
        5003 | Lauson Hen | San Jose |       0.12
Sample table: customer

 customer_id |   cust_name    |    city    | grade | salesman_id
-------------+----------------+------------+-------+-------------
        3002 | Nick Rimando   | New York   |   100 |        5001
        3007 | Brad Davis     | New York   |   200 |        5001
        3005 | Graham Zusi    | California |   200 |        5002
        3008 | Julian Green   | London     |   300 |        5002
        3004 | Fabian Johnson | Paris      |   300 |        5006
        3009 | Geoff Cameron  | Berlin     |   100 |        5003
        3003 | Jozy Altidor   | Moscow     |   200 |        5007
        3001 | Brad Guzan     | London     |       |        5005

2. From the following tables write a SQL query to find those orders where the order amount exists between 500 and 2000. Return 

Sample table: customer

 customer_id |   cust_name    |    city    | grade | salesman_id
```

3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002
3008	Julian Green	London	300	5002
3004	Fabian Johnson	Paris	300	5006
3009	Geoff Cameron	Berlin	100	5003
3003	Jozy Altidor	Moscow	200	5007
3001	Brad Guzan	London		5005

Sample table: orders

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.5	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.5	2012-08-17	3009	5003
70007	948.5	2012-09-10	3005	5002
70005	2400.6	2012-07-27	3007	5001
70008	5760	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.4	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.6	2012-04-25	3002	5001

3. From the following tables write a SQL query to find the salesperson(s) and the customer(s) he represents. Return Customer Na

Sample table: customer

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002
3008	Julian Green	London	300	5002
3004	Fabian Johnson	Paris	300	5006
3009	Geoff Cameron	Berlin	100	5003
3003	Jozy Altidor	Moscow	200	5007
3001	Brad Guzan	London		5005

Sample table: salesman

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13
5003	Lauson Hen	San Jose	0.12

Click me to see the solution with visual presentation

4. From the following tables write a SQL query to find salespeople who received commissions of more than 12 percent from the co

Sample table: customer

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002
3008	Julian Green	London	300	5002
3004	Fabian Johnson	Paris	300	5006
3009	Geoff Cameron	Berlin	100	5003
3003	Jozy Altidor	Moscow	200	5007
3001	Brad Guzan	London		5005

Sample table: salesman

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13
5003	Lauson Hen	San Jose	0.12

Click me to see the solution with visual presentation

5. From the following tables write a SQL query to locate those salespeople who do not live in the same city where their customer

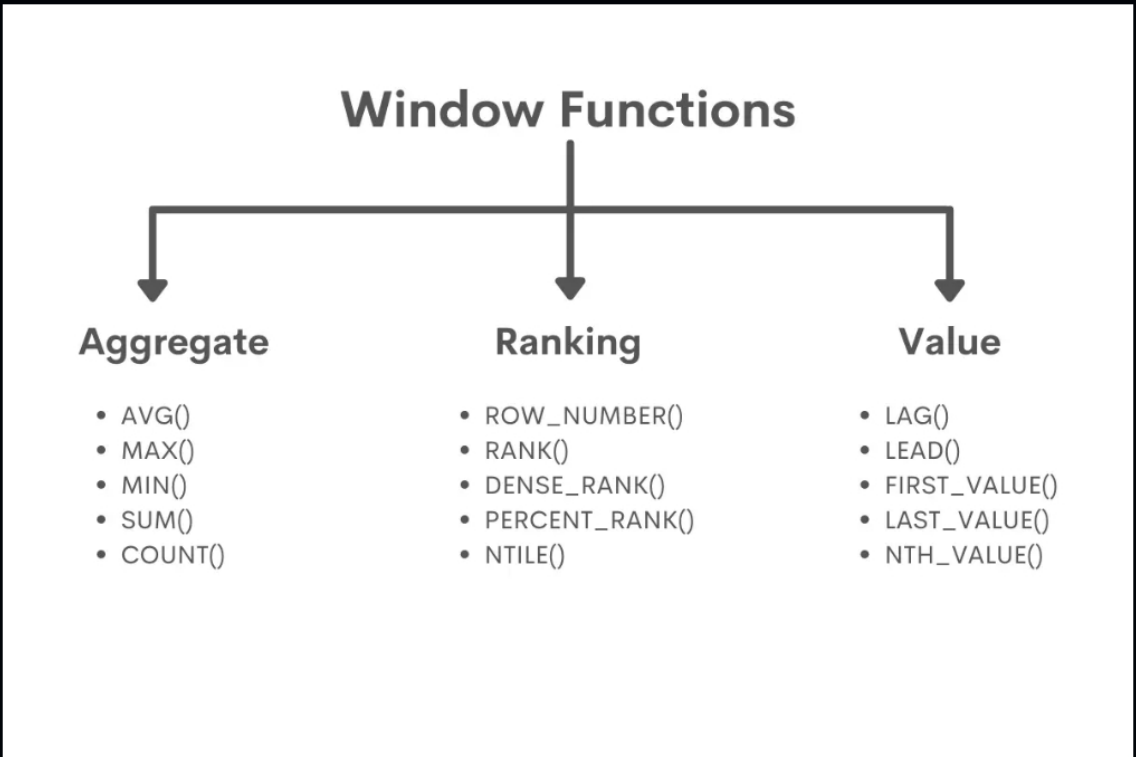
Sample table: customer

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002
3008	Julian Green	London	300	5002
3004	Fabian Johnson	Paris	300	5006
3009	Geoff Cameron	Berlin	100	5003
3003	Jozy Altidor	Moscow	200	5007

```
       3001 | Brad Guzan     | London     |        |        5005
Sample table: salesman

 salesman_id |    name    |   city   | commission
-------------+------------+----------+------------
       5001 | James Hoog | New York |       0.15
       5002 | Nail Knite | Paris    |       0.13
       5005 | Pit Alex   | London   |       0.11
       5006 | Mc Lyon    | Paris    |       0.14
       5007 | Paul Adam  | Rome     |       0.13
       5003 | Lauson Hen | San Jose |       0.12
```

SQL Functions



Aggregate Functions

Perform calculations on a set of values and return a single value.

1. COUNT(): Returns the number of rows.
2. SUM(): Returns the sum of values.
3. AVG(): Returns the average value.
4. MIN(): Returns the minimum value.
5. MAX(): Returns the maximum value.

```sql
SELECT COUNT(*), SUM(amount), AVG(amount) FROM table_name;
```

GROUP BY Clause

The GROUP BY clause is used to group rows that have the same values in one or more columns.

- It is often used in combination with aggregate functions such as SUM, COUNT, MAX, MIN, and AVG.

```sql
-- Syntax:
SELECT column1, column2
FROM tablename
GROUP BY column1, column2;

-- Example:
-- Grouping by one column
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department;

-- Grouping by multiple columns
SELECT department, job_title, AVG(salary) AS average_salary
FROM employees
```

```sql
GROUP BY department, job_title;

-- Using GROUP BY with HAVING clause
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;

-- Using GROUP BY with ORDER BY clause
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department
ORDER BY average_salary DESC;
```

Ranking Functions

Return a ranking or position of a row within a result set.

1. RANK(): Returns the rank of a row within a result set.
2. DENSE_RANK(): Returns the rank of a row within a result set, without gaps.
3. ROW_NUMBER(): Returns a unique number for each row within a result set.
4. NTILE(): Divides a result set into a specified number of groups.

```sql
-- Example of all ranking function in sql
SELECT
    column1,
    column2,
    RANK() OVER (ORDER BY column1) AS rank,
    DENSE_RANK() OVER (ORDER BY column1) AS dense_rank,
    ROW_NUMBER() OVER (ORDER BY column1) AS row_num,
    NTILE(4) OVER (ORDER BY column1) AS ntile
FROM
    table_name;
```

Value Functions

Return a value based on a condition or expression.

1. IF(): Returns one value if the condition is true and another value if the condition is false
2. CASE(): Returns one value if the condition is true and another value if the condition is false
3. COALESCE(): Returns the first non-null value from a list of values.
4. NULLIF(): Returns NULL if the two expressions are equal, otherwise returns the first expression.

Example of value function in sql

```sql
SELECT
    IFNULL(column1, 'default_value') AS column1,
    CASE
        WHEN column2 > 10 THEN 'Greater than 10'
        WHEN column2 < 10 THEN 'Less than 10'
        ELSE 'Equal to 10'
    END AS column2,
    COALESCE(column3, 'default_value') AS column3,
    NULLIF(column4, 'value_to_be_replaced') AS column4
FROM
    table_name;
```

String Functions

Operate on string data types.

1. `CONCAT()` : Concatenates two or more strings.
2. `SUBSTRING()` : Extracts a part of a string.
3. `UPPER()` : Converts a string to uppercase.
4. `LOWER()` : Converts a string to lowercase.
5. `LENGTH()` : Returns the length of a string.
6. `TRIM()` : Removes leading and trailing spaces from a string.
7. `LTRIM()` : Removes leading spaces from a string.
8. `RTRIM()` : Removes trailing spaces from a string.
9. `REPLACE()` : Replaces a substring with another substring.

```sql
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

SELECT
    SUBSTRING_INDEX(employee_name, ' ', 1) AS first_name,
    SUBSTRING_INDEX(employee_name, ' ', -1) AS last_name,
    REPLACE(email, '@', '#') AS modified_email
```

```
FROM employees;
```

```sql
-- Practice question on string, date, and aggregate functions in MySQL

-- Create a table named 'employees' with the following columns:
-- employee_id (primary key, auto-incrementing integer)
-- employee_name (varchar(255))
-- department (varchar(255))
-- salary (integer, default value 50000)
-- hire_date (date)

CREATE TABLE employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_name VARCHAR(255),
    department VARCHAR(255),
    salary INT DEFAULT 50000,
    hire_date DATE
);

-- Insert data into the 'employees' table
INSERT INTO employees (employee_name, department, salary, hire_date) VALUES
('John Doe', 'Sales', 60000, '2020-01-01'),
('Jane Smith', 'Marketing', 55000, '2020-02-01'),
('Bob Johnson', 'IT', 70000, '2020-03-01'),
('Alice Brown', 'HR', 50000, '2020-04-01'),
('Mike Davis', 'Finance', 65000, '2020-05-01');

-- Use string functions to extract the first name and last name from the employee_name column
SELECT
    SUBSTRING_INDEX(employee_name, ' ', 1) AS first_name,
    SUBSTRING_INDEX(employee_name, ' ', -1) AS last_name
FROM employees;

-- Use date functions to calculate the number of years each employee has been with the company
SELECT
    employee_name,
    TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years_with_company
FROM employees;

-- Use aggregate functions to calculate the average salary by department
SELECT
    department,
    AVG(salary) AS average_salary
FROM employees
GROUP BY department;

-- Use aggregate functions to calculate the total salary by department
SELECT
    department,
    SUM(salary) AS total_salary
FROM employees
GROUP BY department;

-- Use aggregate functions to calculate the maximum salary by department
SELECT
    department,
    MAX(salary) AS max_salary
FROM employees
GROUP BY department;

-- Use aggregate functions to calculate the minimum salary by department
SELECT
    department,
    MIN(salary) AS min_salary
FROM employees
GROUP BY department;
```

Date Functions

Work with date and time values.

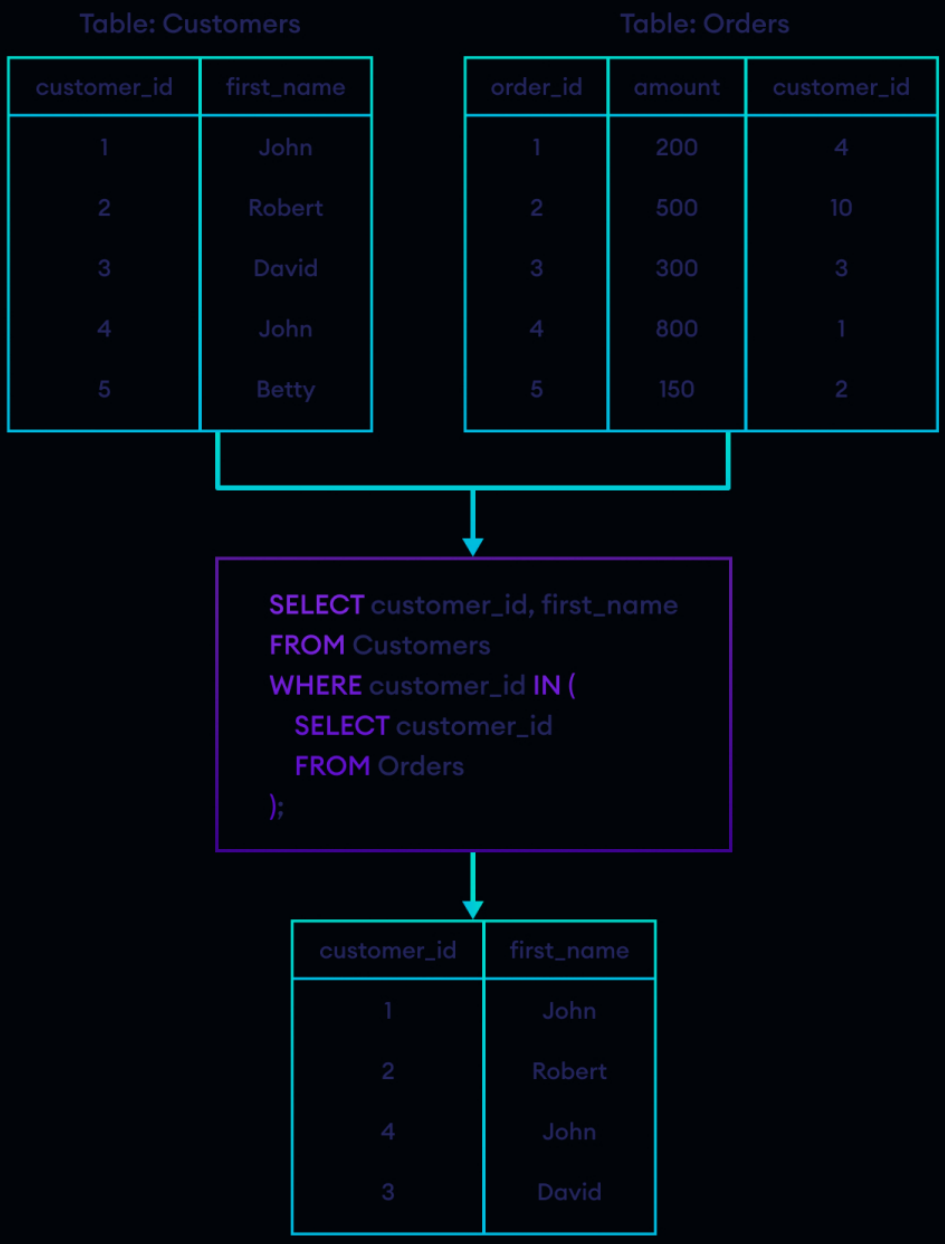
1. `NOW()` : Returns the current date and time.
2. `DATE()` : Extracts the date part.
3. `YEAR()` : Extracts the year part.
4. `MONTH()` : Extracts the month part.
5. `DAY()` : Extracts the day part.
6. `HOUR()` : Extracts the hour part.
7. `MINUTE()` : Extracts the minute part.
8. `SECOND()` : Extracts the second part.
9. `ADDDATE()` : Adds a specified interval to a date.
10. `SUBDATE()` : Subtracts a specified interval from a date.
11. `DATEDIFF()` : Returns the difference between two dates.
12. `DATE_FORMAT()` : Formats a date according to a specified format.

```sql
SELECT NOW(), YEAR(date_column) FROM table_name;
```

Example of all Date Functions

```sql
SELECT
NOW() AS current_date_time,
DATE(date_column) AS date_only,
YEAR(date_column) AS year_only,
MONTH(date_column) AS month_only,
DAY(date_column) AS day_only,
HOUR(date_column) AS hour_only,
MINUTE(date_column) AS minute_only,
SECOND(date_column) AS second_only,
ADDDATE(date_column, INTERVAL 1 DAY) AS add_day,
SUBDATE(date_column, INTERVAL 1 DAY) AS sub_day,
DATEDIFF(date_column, '2020-01-01') AS date_diff,
DATE_FORMAT(date_column, '%Y-%m-%d') AS date_format
FROM table_name;
```

Subqueries



Single-row Subqueries

Return a single row of results.

```
SELECT column1
FROM table_name
WHERE column2 = (SELECT column2 FROM table_name WHERE condition);
```

Example

```
SELECT employee_id
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Multiple-row Subqueries

Return multiple rows of results.

```
SELECT column1
FROM table_name
WHERE column2 IN (SELECT column2 FROM table_name WHERE condition);
```

Example

```
SELECT employee_id
FROM employees
WHERE salary IN (SELECT salary FROM employees WHERE department_id = 1);
```

Transactions

Transactions ensure data integrity by grouping multiple SQL operations.

COMMIT

Saves all changes made in the current transaction.

```
COMMIT;
```

ROLLBACK

Reverts all changes made in the current transaction.

```
ROLLBACK;
```

SAVEPOINT

Sets a point within a transaction to which you can later roll back.

```
START TRANSACTION;
SAVEPOINT savepoint_name;
ROLLBACK TO SAVEPOINT savepoint_name;
```

Example creation of savepoint and use

```
START TRANSACTION;
SAVEPOINT my_savepoint;
-- Make some changes
INSERT INTO my_table (column1, column2) VALUES ('value1', 'value2');
-- Rollback to the savepoint
ROLLBACK TO SAVEPOINT my_savepoint;
```

Indexes

Indexes improve query performance by allowing faster data retrieval.

Creating Indexes

```
CREATE INDEX index_name ON table_name (column_name);
```

Dropping Indexes

```
DROP INDEX index_name ON table_name;
```

Alter Indexes

```
ALTER INDEX index_name ON table_name RENAME TO new_index_name;
```